

Объявление и определение функций, параметры по значению/ссылке, возвращаемые значения, рекурсия - Исчерпывающее руководство по указателям и ссылкам в C++

Основные понятия

Что такое указатель?

Указатель - это переменная, которая хранит **адрес памяти** другой переменной.

Что такое ссылка?

Ссылка - это псевдоним (алиас) для существующей переменной.

Часть 1: Указатели - полный синтаксис

1.1 Базовый синтаксис указателей

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;           // обычная переменная
    int* ptr = &x;       // указатель на int
```

```

cout << "Значение x: " << x << endl;           // 10
cout << "Адрес x: " << &x << endl;             // 0x7fff..
cout << "Значение ptr: " << ptr << endl;        // 0x7fff..
cout << "Адрес ptr: " << &ptr << endl;          // адрес ca
cout << "Значение по адресу ptr: " << *ptr << endl; // 10

return 0;
}

```

1.2 Операторы работы с указателями

```

#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int b = 10;

    // 1. Оператор взятия адреса (&)
    int* ptr_a = &a; // ptr_a хранит адрес a
    int* ptr_b = &b; // ptr_b хранит адрес b

    // 2. Оператор разыменования (*)
    *ptr_a = 20;      // изменяем значение a через указатель
    cout << "a = " << a << endl; // 20

    // 3. Арифметика указателей
    int arr[5] = {10, 20, 30, 40, 50};
    int* arr_ptr = arr; // указатель на первый элемент

    cout << "\nАрифметика указателей:" << endl;
    cout << "arr_ptr = " << arr_ptr << endl;
    cout << "*arr_ptr = " << *arr_ptr << endl; // 10

    arr_ptr++; // переход к следующему элементу
    cout << "После arr_ptr++:" << endl;
    cout << "arr_ptr = " << arr_ptr << endl;
    cout << "*arr_ptr = " << *arr_ptr << endl; // 20

    arr_ptr += 2; // сдвиг на 2 элемента вперед
}

```

```

cout << "После arr_ptr += 2:" << endl;
cout << "*arr_ptr = " << *arr_ptr << endl;  // 40

arr_ptr--;  // сдвиг на 1 элемент назад
cout << "После arr_ptr--:" << endl;
cout << "*arr_ptr = " << *arr_ptr << endl;  // 30

// Разница между указателями
int* ptr1 = &arr[0];
int* ptr2 = &arr[3];
cout << "\nРазница ptr2 - ptr1 = " << (ptr2 - ptr1) << " :

return 0;
}

```

1.3 Указатель на указатель (двойной указатель)

```

#include <iostream>
using namespace std;

int main() {
    int x = 42;
    int* ptr = &x;      // указатель на int
    int** ptr_ptr = &ptr; // указатель на указатель на int

    cout << "x = " << x << endl;           // 42
    cout << "&x = " << &x << endl;         // адрес x
    cout << "ptr = " << ptr << endl;        // адрес x
    cout << "*ptr = " << *ptr << endl;       // 42

    cout << "\nptr_ptr = " << ptr_ptr << endl; // адрес ptr
    cout << "*ptr_ptr = " << *ptr_ptr << endl; // адрес x
    cout << "**ptr_ptr = " << **ptr_ptr << endl; // 42

    // Изменение через двойной указатель
    **ptr_ptr = 100;
    cout << "\nПосле **ptr_ptr = 100:" << endl;
    cout << "x = " << x << endl;           // 100

    // Тройной указатель
    int*** ptr_ptr_ptr = &ptr_ptr;
}

```

```

***ptr_ptr_ptr = 200;
cout << "После ***ptr_ptr_ptr = 200:" << endl;
cout << "x = " << x << endl;           // 200

return 0;
}

```

1.4 Указатель на массив vs массив указателей

```

#include <iostream>
using namespace std;

int main() {
    // 1. Указатель на массив
    int arr[5] = {1, 2, 3, 4, 5};
    int (*ptr_to_array)[5] = &arr; // указатель на массив из

    cout << "Указатель на массив:" << endl;
    cout << "arr = " << arr << endl;           // адрес
    cout << "&arr = " << &arr << endl;         // адрес
    cout << "ptr_to_array = " << ptr_to_array << endl;

    // Доступ через указатель на массив
    for(int i = 0; i < 5; i++) {
        cout << (*ptr_to_array)[i] << " "; // доступ через ptr
    }
    cout << endl;

    // 2. Массив указателей
    int x = 10, y = 20, z = 30;
    int* array_of_pointers[3] = {&x, &y, &z};

    cout << "\nМассив указателей:" << endl;
    for(int i = 0; i < 3; i++) {
        cout << "array_of_pointers[" << i << "] = " << array_of_pointers[i] << endl;
        cout << ", * = " << *array_of_pointers[i] << endl;
    }

    // 3. Двумерный массив через указатели
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int (*ptr_to_matrix)[3] = matrix; // указатель на массив

```

```

    cout << "\nДоступ к матрице через указатель:" << endl;
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            cout << ptr_to_matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}

```

1.5 Указатели на функции

```

#include <iostream>
using namespace std;

// Объявление функций
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
double divide(int a, int b) { return static_cast<double>(a) /

int main() {
    // Указатель на функцию, возвращающую int и принимающую два
    int (*func_ptr)(int, int);

    func_ptr = add; // присваиваем адрес функции add
    cout << "add(5, 3) = " << func_ptr(5, 3) << endl; // 8

    func_ptr = subtract;
    cout << "subtract(5, 3) = " << func_ptr(5, 3) << endl; //

    func_ptr = multiply;
    cout << "multiply(5, 3) = " << func_ptr(5, 3) << endl; //

    // Массив указателей на функции
    int (*operations[3])(int, int) = {add, subtract, multiply};
    char* op_names[3] = {"Сложение", "Вычитание", "Умножение"};

    cout << "\nМассив указателей на функции:" << endl;
}

```

```

for(int i = 0; i < 3; i++) {
    cout << op_names[i] << ": " << operations[i](10, 5) << endl;
}

// Указатель на функцию с другим типом возвращаемого значения
double (*div_ptr)(int, int) = divide;
cout << "\nДеление: 10 / 3 = " << div_ptr(10, 3) << endl;

// typedef для упрощения
typedef int (*MathOperation)(int, int);
MathOperation op1 = add;
MathOperation op2 = multiply;

cout << "\nИспользование typedef:" << endl;
cout << "op1(7, 2) = " << op1(7, 2) << endl; // 9
cout << "op2(7, 2) = " << op2(7, 2) << endl; // 14

return 0;
}

```

1.6 const с указателями (4 варианта!)

```

#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int y = 20;

    cout << "=== 4 варианта const с указателями ===" << endl;

    // 1. Указатель на константу (pointer to const)
    // Значение нельзя изменить через указатель, но указатель
    const int* ptr_to_const = &x;
    // *ptr_to_const = 30; // ОШИБКА: нельзя изменить значение
    ptr_to_const = &y; // ОК: можно изменить сам указатель
    cout << "1. Указатель на константу: *ptr = " << *ptr_to_const << endl;

    // 2. Константный указатель (const pointer)
    // Указатель нельзя перенаправить, но значение можно изменить
    int* const const_ptr = &x;

```

```

*const_ptr = 30;          // ОК: можно изменить значение
// const_ptr = &y;        // ОШИБКА: нельзя изменить указате
cout << "2. Константный указатель: *ptr = " << *const_ptr

// 3. Константный указатель на константу (const pointer to
// Ничего нельзя изменить
const int* const const_ptr_to_const = &x;
// *const_ptr_to_const = 40; // ОШИБКА
// const_ptr_to_const = &y;   // ОШИБКА
cout << "3. Константный указатель на константу: *ptr = "
    << *const_ptr_to_const << endl;

// 4. Указатель на константный указатель на константу
const int* const* complex_ptr = &const_ptr_to_const;
cout << "4. Сложный указатель: **ptr = " << **complex_ptr

// Практический пример
cout << "\n=== Практический пример ===" << endl;
const int data[] = {1, 2, 3, 4, 5};
const int* data_ptr = data; // указатель на константу

// Перебор массива
for(int i = 0; i < 5; i++) {
    cout << "data[" << i << "] = " << *(data_ptr + i) << " ";
    // *(data_ptr + i) = 0; // ОШИБКА: данные константны
}

return 0;
}

```

1.7 void указатели (универсальные указатели)

```

#include <iostream>
using namespace std;

int main() {
    int x = 42;
    double y = 3.14;
    char z = 'A';

    // void* может указывать на данные любого типа

```

```
void* void_ptr;

void_ptr = &x; // указывает на int
cout << "int value: " << *(static_cast<int*>(void_ptr)) << endl;

void_ptr = &y; // указывает на double
cout << "double value: " << *(static_cast<double*>(void_ptr)) << endl;

void_ptr = &z; // указывает на char
cout << "char value: " << *(static_cast<char*>(void_ptr)) << endl;

// Пример: функция для вывода значения любого типа
cout << "\nФункция printAnyValue:" << endl;

// Нельзя разыменовать void* без приведения типа!
// cout << *void_ptr; // ОШИБКА

// Массив void указателей
void* values[3];
values[0] = &x;
values[1] = &y;
values[2] = &z;

char* types[3] = {"int", "double", "char"};

for(int i = 0; i < 3; i++) {
    cout << "Тип " << types[i] << ": ";

    if(types[i] == string("int")) {
        cout << *(static_cast<int*>(values[i])) << endl;
    }
    else if(types[i] == string("double")) {
        cout << *(static_cast<double*>(values[i])) << endl;
    }
    else if(types[i] == string("char")) {
        cout << *(static_cast<char*>(values[i])) << endl;
    }
}

return 0;
}
```

1.8 Указатели и динамическая память

```
#include <iostream>
using namespace std;

int main() {
    cout << "=== Динамическое выделение памяти ===" << endl;

    // 1. Выделение памяти для одного значения
    int* single_ptr = new int(42);
    cout << "Динамическое int: " << *single_ptr << endl;

    // 2. Выделение памяти для массива
    int size = 5;
    int* array_ptr = new int[size];

    // Заполнение массива
    for(int i = 0; i < size; i++) {
        array_ptr[i] = i * 10;
    }

    cout << "Динамический массив: ";
    for(int i = 0; i < size; i++) {
        cout << array_ptr[i] << " ";
    }
    cout << endl;

    // 3. Изменение размера массива
    int new_size = 8;
    int* new_array_ptr = new int[new_size];

    // Копирование старых данных
    for(int i = 0; i < size && i < new_size; i++) {
        new_array_ptr[i] = array_ptr[i];
    }

    // Освобождение старой памяти
    delete[] array_ptr;
    array_ptr = new_array_ptr;
    size = new_size;

    cout << "После увеличения размера: ";
```

```
for(int i = 0; i < size; i++) {
    cout << array_ptr[i] << " ";
}
cout << endl;

// 4. Двумерный динамический массив
int rows = 3, cols = 4;
int** matrix = new int*[rows];

for(int i = 0; i < rows; i++) {
    matrix[i] = new int[cols];
    for(int j = 0; j < cols; j++) {
        matrix[i][j] = i * 10 + j;
    }
}

cout << "\nДинамическая матрица " << rows << "x" << cols .
for(int i = 0; i < rows; i++) {
    for(int j = 0; j < cols; j++) {
        cout << matrix[i][j] << "\t";
    }
    cout << endl;
}

// 5. ОСВОБОЖДЕНИЕ ПАМЯТИ (ВАЖНО!)
delete single_ptr;      // для одиночного значения
delete[] array_ptr;     // для массива

// Для двумерного массива
for(int i = 0; i < rows; i++) {
    delete[] matrix[i];
}
delete[] matrix;

// 6. Указатель после delete
int* ptr = new int(100);
delete ptr;
ptr = nullptr; // хорошая практика

// Проверка на nullptr перед использованием
if(ptr != nullptr) {
    cout << "Значение: " << *ptr << endl;
} else {
```

```
        cout << "Указатель nullptr" << endl;
    }

    return 0;
}
```

Часть 2: Ссылки - полный синтаксис

2.1 Базовый синтаксис ссылок

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;

    // Объявление ссылки (должна быть инициализирована сразу)
    int& ref = x; // ref - это псевдоним для x

    cout << "x = " << x << endl;        // 10
    cout << "ref = " << ref << endl;      // 10
    cout << "&x = " << &x << endl;      // адрес x
    cout << "&ref = " << &ref << endl;   // тот же адрес

    // Изменение через ссылку изменяет оригинальную переменную
    ref = 20;
    cout << "\nПосле ref = 20:" << endl;
    cout << "x = " << x << endl;        // 20
    cout << "ref = " << ref << endl;     // 20

    // Ссылка не может быть перенаправлена!
    int y = 30;
    // ref = y; // Это присвоение значения, не перенаправление
    ref = y;    // x = 30, ref = 30, но ref все еще ссылается
    cout << "\nПосле ref = y (y=30):" << endl;
    cout << "x = " << x << endl;        // 30
    cout << "ref = " << ref << endl;     // 30
    cout << "y = " << y << endl;        // 30
}
```

```

        // Можно создать ссылку на ссылку (фактически, ссылку на
        int& ref_ref = ref; // ref_ref тоже ссылается на x
        ref_ref = 40;
        cout << "\nПосле ref_ref = 40:" << endl;
        cout << "x = " << x << endl;          // 40

        return 0;
    }

```

2.2 Ссылки как параметры функций

```

#include <iostream>
using namespace std;

// 1. Параметр по значению (копия)
void incrementByValue(int x) {
    x++;
    cout << "Внутри incrementByValue: x = " << x << endl;
}

// 2. Параметр по ссылке (работа с оригиналом)
void incrementByReference(int& x) {
    x++;
    cout << "Внутри incrementByReference: x = " << x << endl;
}

// 3. Константная ссылка (чтение без копирования)
void printValue(const int& x) {
    // x = 100; // ОШИБКА: нельзя изменить константную ссылку
    cout << "Внутри printValue: x = " << x << endl;
}

// 4. Возврат ссылки (ОПАСНО, если возвращается ссылка на локальную переменную)
int& getElement(int arr[], int index) {
    return arr[index]; // ОК: возвращаем ссылку на элемент массива
}

// НЕПРАВИЛЬНО: возврат ссылки на локальную переменную
int& badFunction() {
    int x = 100; // локальная переменная
    return x;    // ОШИБКА: x уничтожится после выхода
}

```

```

}

int main() {
    int a = 5;

    cout << "До incrementByValue: a = " << a << endl;
    incrementByValue(a);
    cout << "После incrementByValue: a = " << a << endl;    // 1

    cout << "\nДо incrementByReference: a = " << a << endl;
    incrementByReference(a);
    cout << "После incrementByReference: a = " << a << endl;

    cout << "\nВызов printValue: ";
    printValue(a);

    // Работа с возвращаемой ссылкой
    int arr[5] = {10, 20, 30, 40, 50};
    getElement(arr, 2) = 999;    // изменяем элемент через возвра

    cout << "\nПосле getElement(arr, 2) = 999:" << endl;
    cout << "arr[2] = " << arr[2] << endl;    // 999

    return 0;
}

```

2.3 Ссылки на массивы

```

#include <iostream>
using namespace std;

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    // 1. Ссылка на массив фиксированного размера
    int (&ref_to_array)[5] = arr;

    cout << "Ссылка на массив:" << endl;
    for(int i = 0; i < 5; i++) {
        cout << "ref_to_array[" << i << "] = " << ref_to_array[i] << " ";
    }
}

```

```

// 2. Изменение через ссылку
ref_to_array[0] = 100;
cout << "\nПосле ref_to_array[0] = 100:" << endl;
cout << "arr[0] = " << arr[0] << endl; // 100

// 3. Ссылка на массив как параметр функции
auto printArray = [](int (&array)[5]) {
    cout << "В функции: ";
    for(int i = 0; i < 5; i++) {
        cout << array[i] << " ";
    }
    cout << endl;
};

printArray(arr);

// 4. Ссылка на двумерный массив
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
int (&ref_to_matrix)[2][3] = matrix;

cout << "\nСсылка на матрицу:" << endl;
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 3; j++) {
        cout << ref_to_matrix[i][j] << " ";
    }
    cout << endl;
}

return 0;
}

```

2.4 const со ссылками

```

#include <iostream>
using namespace std;

int main() {
    int x = 10;
    const int y = 20;

```

```

cout << "=== const со ссылками ===" << endl;

// 1. Обычная ссылка на неконстантную переменную
int& ref1 = x;          // ОК
// int& ref2 = y;      // ОШИБКА: нельзя ссылаться на const

// 2. Константная ссылка на неконстантную переменную
const int& cref1 = x;   // ОК: доступ только для чтения
cout << "cref1 = " << cref1 << endl;
// cref1 = 30;         // ОШИБКА: нельзя изменить через const

// 3. Константная ссылка на константную переменную
const int& cref2 = y;   // ОК
cout << "cref2 = " << cref2 << endl;

// 4. Временные объекты и const ссылки
const int& temp_ref = 42; // ОК: const ссылка может ссылаться на временный объект
cout << "temp_ref = " << temp_ref << endl;

// int& bad_temp_ref = 42; // ОШИБКА: не-const ссылка не может ссылаться на временный объект

// 5. Константная ссылка продлевает время жизни временного объекта
const int& extended_life = x + y; // временный объект живет дольше
cout << "extended_life = " << extended_life << endl;

// Практический пример
cout << "\n=== Практический пример ===" << endl;
int a = 5, b = 3;

// Функция, возвращающая константную ссылку на больший аргумент
auto& max_ref = (a > b) ? a : b;
const auto& max_cref = (a > b) ? a : b;

max_ref = 100; // можно изменить
// max_cref = 100; // ОШИБКА

cout << "a = " << a << ", b = " << b << endl;

return 0;
}

```

2.5 Ссылки на указатели и указатели на ссылки

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int y = 20;

    cout << "=== Полный разбор ссылок на указатели ===" << endl;
    cout << "x = " << x << ", y = " << y << endl << endl;

    // 1. Базовые типы указателей
    cout << "1. Базовые типы указателей:" << endl;
    cout << "-----" << endl;

    int* ptr = &x;                // указатель на int
    const int* ptr_to_const = &x; // указатель на const int
    int* const const_ptr = &x;    // константный указатель
    const int* const const_ptr_to_const = &x; // константный

    cout << "int* ptr: можно изменить и указатель, и значение" << endl;
    cout << "const int* ptr_to_const: можно изменить указатель" << endl;
    cout << "int* const const_ptr: НЕЛЬЗЯ изменить указатель," << endl;
    cout << "const int* const: НЕЛЬЗЯ изменить ни указатель, ни значение" << endl;

    // 2. Ссылки на разные типы указателей
    cout << "2. Ссылки на указатели:" << endl;
    cout << "-----" << endl;

    // а) Ссылка на обычный указатель
    int*& ref_to_ptr = ptr;
    cout << "int*& ref_to_ptr = ptr;" << endl;
    cout << "  *ptr = " << *ptr << ", *ref_to_ptr = " << *ref_to_ptr << endl;

    // Меняем указатель через ссылку
    ref_to_ptr = &y;
    cout << "  После ref_to_ptr = &y:" << endl;
    cout << "  *ptr = " << *ptr << ", *ref_to_ptr = " << *ref_to_ptr << endl;
    cout << "  y = " << y << endl;

    // Возвращаем указатель обратно на x
    ref_to_ptr = &x;
    cout << "  После ref_to_ptr = &x:" << endl;
    cout << "  *ptr = " << *ptr << ", *ref_to_ptr = " << *ref_to_ptr << endl;
    cout << "  x = " << x << endl;
}
```

```

ptr = &x;

// б) Ссылка на указатель на константу
const int* ptr2 = &x;
const int*& ref_to_ptr_to_const = ptr2; // OK!
cout << "\nconst int*& ref_to_ptr_to_const = ptr2;" << endl;
cout << "  *ptr2 = " << *ptr2 << ", *ref_to_ptr_to_const :

// Можно изменить указатель, но не значение
ptr2 = &y;
cout << "  После ptr2 = &y:" << endl;
cout << "  *ptr2 = " << *ptr2 << endl;
// *ptr2 = 50; // ОШИБКА!

// в) Ссылка на константный указатель
int* const const_ptr2 = &x;
int* const& ref_to_const_ptr = const_ptr2; // OK!
cout << "\nint* const& ref_to_const_ptr = const_ptr2;" << endl;
cout << "  *const_ptr2 = " << *const_ptr2 << endl;

// Можно изменить значение, но не указатель
*const_ptr2 = 100;
cout << "  После *const_ptr2 = 100:" << endl;
cout << "  x = " << x << endl;
// const_ptr2 = &y; // ОШИБКА!

// г) Почему была ошибка в исходном коде?
cout << "\n3. Разбор ошибки:" << endl;
cout << "-----" << endl;
cout << "Исходная ошибка: const int*& ref = ptr;" << endl;
cout << "Где ptr имеет тип int*" << endl << endl;

cout << "Проблема: мы пытаемся создать ссылку, которая обн
cout << "что через нее нельзя изменить значение (const in
cout << "привязываем ее к указателю, который МОЖЕТ изменят

cout << "Аналогия: обещаем не менять книгу (const), но да
cout << "на полку, где лежит обычная книга (не const)." <<
cout << "Кто-то другой может через исходный указатель изм
cout << "и наше обещание будет нарушено." << endl;

// 4. Правильный способ работы с разными типами
cout << "\n4. Правильные преобразования:" << endl;

```

```

cout << "-----" << endl;

// Можно преобразовать int* в const int*, но не наоборот
const int* safe_ptr = ptr; // OK: добавляем константности
cout << "const int* safe_ptr = ptr; // OK" << endl;
cout << "*safe_ptr = " << *safe_ptr << endl;

// А вот ссылки более строгие:
// const int*& ref = ptr; // ОШИБКА!
// Но можно через промежуточную переменную:
const int* const_ptr3 = ptr;
const int*& ref = const_ptr3; // OK
cout << "\nЧерез промежуточную переменную:" << endl;
cout << "const int* const_ptr3 = ptr;" << endl;
cout << "const int*& ref = const_ptr3; // OK" << endl;

// 5. Указатель на указатель и ссылки на них
cout << "\n5. Указатели на указатели:" << endl;
cout << "-----" << endl;

int** pp = &ptr; // указатель на указатель
int**& ref_to_pp = pp; // ссылка на указатель на указатель
int*** ppp = &pp; // указатель на указатель на указатель

cout << "int** pp = &ptr;" << endl;
cout << "int**& ref_to_pp = pp;" << endl;
cout << "int*** ppp = &pp;" << endl << endl;

// Работа с двойным разыменованием
cout << "Значения:" << endl;
cout << " x = " << x << endl;
cout << " *ptr = " << *ptr << endl;
cout << " **pp = " << **pp << endl;
cout << " **ref_to_pp = " << **ref_to_pp << endl;

// Меняем x через двойной указатель
**pp = 999;
cout << "\n После **pp = 999:" << endl;
cout << " x = " << x << endl;
cout << " *ptr = " << *ptr << endl;
cout << " **pp = " << **pp << endl;

```

```
    return 0;  
}
```

2.6 Rvalue-ссылки (C++11 и выше)

```
#include <iostream>  
#include <utility> // для std::move  
using namespace std;  
  
int main() {  
    cout << "=== Rvalue-ссылки (C++11) ===" << endl;  
  
    int x = 10;  
  
    // Lvalue-ссылка (обычная ссылка)  
    int& lref = x; // OK: x - lvalue  
  
    // Rvalue-ссылка (ссылка на временное значение)  
    int&& rref = 20; // OK: 20 - rvalue  
    cout << "rref = " << rref << endl; // 20  
  
    // Rvalue-ссылка на временный объект  
    int&& rref2 = x + 5; // OK: x+5 - rvalue  
    cout << "rref2 = " << rref2 << endl; // 15  
  
    // Нельзя создать lvalue-ссылку на rvalue  
    // int& bad_ref = 30; // ОШИБКА  
  
    // Пример с std::move  
    cout << "\nИспользование std::move:" << endl;  
  
    string str1 = "Hello";  
    string str2 = "World";  
  
    cout << "До move:" << endl;  
    cout << "str1 = " << str1 << endl;  
    cout << "str2 = " << str2 << endl;  
  
    // Перемещение содержимого (без копирования)  
    str2 = std::move(str1);
```

```

cout << "\nПосле str2 = std::move(str1):" << endl;
cout << "str1 = " << str1 << endl; // пустая или в неопределенном состоянии
cout << "str2 = " << str2 << endl; // "Hello"

// Пример функции с rvalue-ссылкой
auto processValue = [](int&& val) {
    cout << "Обработка rvalue: " << val << endl;
};

auto processLValue = [](int& val) {
    cout << "Обработка lvalue: " << val << endl;
};

cout << "\nВызов функций:" << endl;
processValue(42); // OK: 42 - rvalue
// processValue(x); // ОШИБКА: x - lvalue

processLValue(x); // OK: x - lvalue
// processLValue(42); // ОШИБКА: 42 - rvalue

// Универсальная ссылка (шаблонная rvalue-ссылка)
auto universalRef = [](auto&& param) {
    cout << "Универсальная ссылка: " << param << endl;
};

cout << "\nУниверсальные ссылки:" << endl;
universalRef(x); // OK: lvalue
universalRef(42); // OK: rvalue

return 0;
}

```

Часть 3: Сравнение указателей и ссылок

3.1 Основные различия

```

#include <iostream>
using namespace std;

```

```

void comparePointersAndReferences() {
    int x = 10;
    int y = 20;

    cout << "=== Сравнение указателей и ссылок ===" << endl;

    // УКАЗАТЕЛИ:
    int* ptr = &x;
    cout << "\nУказатели:" << endl;
    cout << "1. Могут быть nullptr: " << (ptr == nullptr ? "да" : "нет") << endl;
    cout << "2. Можно перенаправить: ";
    ptr = &y;
    cout << "теперь указывает на y, *ptr = " << *ptr << endl;
    cout << "3. Имеют свой адрес: &ptr = " << &ptr << endl;
    cout << "4. Требуют разыменования: *ptr для доступа к значению" << endl;

    // ССЫЛКИ:
    int& ref = x;
    cout << "\nСсылки:" << endl;
    cout << "1. Не могут быть nullptr" << endl;
    cout << "2. Нельзя перенаправить после инициализации" << endl;
    cout << "3. Не имеют своего адреса: &ref = адрес x = " << &ref << endl;
    cout << "4. Не требуют разыменования: ref напрямую дает значение" << endl;

    // Инициализация:
    cout << "\nИнициализация:" << endl;
    // int* ptr2; // ОК: неинициализированный указатель (опаасно)
    // int& ref2; // ОШИБКА: ссылка должна быть инициализирована

    // Арифметика:
    cout << "\nАрифметика:" << endl;
    cout << "Указатели: можно ptr++, ptr--, ptr + n" << endl;
    cout << "Ссылки: арифметики нет" << endl;

    // Размер:
    cout << "\nРазмер в памяти:" << endl;
    cout << "sizeof(ptr) = " << sizeof(ptr) << " байт" << endl;
    cout << "sizeof(ref) = " << sizeof(ref) << " байт (размер int)" << endl;

    // Массивы:
    cout << "\nРабота с массивами:" << endl;
    int arr[3] = {1, 2, 3};
    int* arr_ptr = arr; // ОК

```

```
    // int& arr_ref = arr;  // ОШИБКА: нужен конкретный синтаксис
    int (&arr_ref)[3] = arr;  // правильный синтаксис
}
```

3.2 Когда что использовать

```
#include <iostream>
#include <vector>
using namespace std;

// ✅ ИСПОЛЬЗОВАТЬ ССЫЛКИ:
// 1. Параметры функций для модификации
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// 2. Для избежания копирования больших объектов
void processVector(const vector<int>& vec) {
    // Без копирования, только чтение
    for(const auto& elem : vec) {
        cout << elem << " ";
    }
    cout << endl;
}

// 3. Возврат элемента из контейнера
int& getElement(vector<int>& vec, size_t index) {
    return vec[index];  // возвращаем ссылку на элемент
}

// ✅ ИСПОЛЬЗОВАТЬ УКАЗАТЕЛИ:
// 1. Динамическое выделение памяти
int* createArray(int size) {
    return new int[size];
}

// 2. Когда параметр может отсутствовать
void printIfNotNull(const int* ptr) {
    if(ptr != nullptr) {
```

```
        cout << "Значение: " << *ptr << endl;
    } else {
        cout << "Указатель равен nullptr" << endl;
    }
}

// 3. Работа с массивами в C-стиле
void processCArray(int* arr, int size) {
    for(int i = 0; i < size; i++) {
        arr[i] *= 2;
    }
}

// 4. Полиморфизм и наследование
class Base {
public:
    virtual void print() { cout << "Base" << endl; }
};

class Derived : public Base {
public:
    void print() override { cout << "Derived" << endl; }
};

void polymorphicExample() {
    Derived d;
    Base* ptr = &d; // указатель для полиморфизма
    ptr->print();    // вызовет Derived::print()
}

int main() {
    cout << "=== Рекомендации по использованию ===" << endl;

    // Пример 1: Ссылки лучше для простых случаев
    int a = 5, b = 10;
    swap(a, b);
    cout << "После swap: a=" << a << ", b=" << b << endl;

    // Пример 2: Указатели для опциональных параметров
    int x = 42;
    printIfNotNull(&x);
    printIfNotNull(nullptr);
}
```

```
// Пример 3: Ссылки для возврата значений
vector<int> vec = {1, 2, 3, 4, 5};
getElement(vec, 2) = 100; // изменяем элемент через ссылку

cout << "\nВектор после изменения: ";
processVector(vec);

// Пример 4: Указатели для динамической памяти
int* dynamic_arr = createArray(5);
for(int i = 0; i < 5; i++) {
    dynamic_arr[i] = i * 10;
}

cout << "\nДинамический массив: ";
for(int i = 0; i < 5; i++) {
    cout << dynamic_arr[i] << " ";
}
cout << endl;

delete[] dynamic_arr;

return 0;
}
```

Часть 4: Продвинутые примеры

4.1 Умные указатели (C++11 и выше)

```
#include <iostream>
#include <memory> // для умных указателей
using namespace std;

class Resource {
public:
    Resource() { cout << "Resource создан" << endl; }
    ~Resource() { cout << "Resource уничтожен" << endl; }
    void use() { cout << "Resource используется" << endl; }
};
```

```

int main() {
    cout << "=== Умные указатели ===" << endl;

    // 1. unique_ptr - эксклюзивное владение
    {
        cout << "\n=== unique_ptr ===" << endl;
        unique_ptr<Resource> ptr1(new Resource());
        // unique_ptr<Resource> ptr2 = ptr1; // ОШИБКА: нельзя копировать
        unique_ptr<Resource> ptr2 = move(ptr1); // можно переместить

        if(ptr1) cout << "ptr1 владеет ресурсом" << endl;
        else cout << "ptr1 не владеет ресурсом" << endl;

        ptr2->use();
    } // ресурс автоматически удаляется

    // 2. shared_ptr - разделяемое владение (счетчик ссылок)
    {
        cout << "\n=== shared_ptr ===" << endl;
        shared_ptr<Resource> ptr1 = make_shared<Resource>();
        {
            shared_ptr<Resource> ptr2 = ptr1; // можно копировать
            cout << "Количество ссылок: " << ptr1.use_count() << endl;

            shared_ptr<Resource> ptr3 = ptr1;
            cout << "Количество ссылок: " << ptr1.use_count() << endl;
        } // ptr2 и ptr3 уничтожаются

        cout << "Количество ссылок: " << ptr1.use_count() << endl;
        ptr1->use();
    } // ресурс удаляется, когда счетчик становится 0

    // 3. weak_ptr - слабая ссылка (не увеличивает счетчик)
    {
        cout << "\n=== weak_ptr ===" << endl;
        shared_ptr<Resource> shared = make_shared<Resource>();
        weak_ptr<Resource> weak = shared;

        cout << "Количество ссылок shared: " << shared.use_count() << endl;

        if(auto temp = weak.lock()) { // создаем shared_ptr из weak_ptr
            cout << "Ресурс еще существует" << endl;
            temp->use();
        }
    }
}

```

```

    }

    shared.reset(); // удаляем ресурс

    if(auto temp = weak.lock()) {
        cout << "Ресурс еще существует" << endl;
    } else {
        cout << "Ресурс уже удален" << endl;
    }
}

// 4. Пользовательский делитер
{
    cout << "\n=== Пользовательский делитер ===" << endl;
    auto custom_deleter = [](Resource* res) {
        cout << "Пользовательский делитер вызывается" << endl;
        delete res;
    };

    unique_ptr<Resource, decltype(custom_deleter)>
        ptr(new Resource(), custom_deleter);
    ptr->use();
}

return 0;
}

```

4.2 Функциональные объекты и указатели на члены класса

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class Person {
public:
    string name;
    int age;

    Person(string n, int a) : name(n), age(a) {}
}

```

```

void print() const {
    cout << name << " (" << age << " лет)" << endl;
}

bool isAdult() const {
    return age >= 18;
}
};

int main() {
    cout << "=== Указатели на члены класса ===" << endl;

    vector<Person> people = {
        Person("Анна", 25),
        Person("Борис", 17),
        Person("Виктор", 30),
        Person("Галина", 16)
    };

    // 1. Указатель на член-данные
    int Person::* age_ptr = &Person::age;
    string Person::* name_ptr = &Person::name;

    cout << "\nДоступ через указатели на члены:" << endl;
    for(auto& person : people) {
        cout << person.*name_ptr << ": " << person.*age_ptr << "
    }

    // 2. Указатель на метод класса
    void (Person::* print_ptr)() const = &Person::print;
    bool (Person::* adult_ptr)() const = &Person::isAdult;

    cout << "\nВызов методов через указатели:" << endl;
    for(auto& person : people) {
        (person.*print_ptr)();
        cout << "Совершеннолетний: " << ((person.*adult_ptr)()) ?
    }

    // 3. Использование с алгоритмами STL
    cout << "\nСовершеннолетние люди:" << endl;
    for(auto& person : people) {
        if((person.*adult_ptr)()) {
            (person.*print_ptr)();
        }
    }
}

```

```

    }
}

// 4. std::function и указатели на методы
cout << "\nИспользование std::function:" << endl;
vector<function<void()>> callbacks;

for(auto& person : people) {
    // Захватываем person по ссылке и вызываем метод чере:
    callbacks.push_back([&person, print_ptr]() {
        (person.*print_ptr)();
    });
}

for(auto& callback : callbacks) {
    callback();
}

return 0;
}

```

4.3 typeid и указатели

```

#include <iostream>
#include <typeinfo> // для typeid
using namespace std;

class Base {
public:
    virtual ~Base() {} // виртуальный деструктор для полимор
};

class Derived : public Base {};

int main() {
    cout << "=== typeid и указатели ===" << endl;

    int x = 10;
    double y = 3.14;
    Base base;
    Derived derived;
}

```

```

// typeid для обычных переменных
cout << "typeid(x) = " << typeid(x).name() << endl;
cout << "typeid(y) = " << typeid(y).name() << endl;
cout << "typeid(base) = " << typeid(base).name() << endl;
cout << "typeid(derived) = " << typeid(derived).name() << endl;

// typeid для указателей (без разыменования)
Base* ptr1 = &base;
Base* ptr2 = &derived;

cout << "\nДля указателей:" << endl;
cout << "typeid(ptr1) = " << typeid(ptr1).name() << endl;
cout << "typeid(ptr2) = " << typeid(ptr2).name() << endl;
cout << "typeid(*ptr1) = " << typeid(*ptr1).name() << endl;
cout << "typeid(*ptr2) = " << typeid(*ptr2).name() << endl;

// Сравнение типов
cout << "\nСравнение типов:" << endl;
cout << "typeid(x) == typeid(int): " << (typeid(x) == typeid(int)) << endl;
cout << "typeid(ptr1) == typeid(Base*): " << (typeid(ptr1) == typeid(Base*)) << endl;
cout << "typeid(*ptr1) == typeid(Base): " << (typeid(*ptr1) == typeid(Base)) << endl;
cout << "typeid(*ptr2) == typeid(Derived): " << (typeid(*ptr2) == typeid(Derived)) << endl;

// Динамическое приведение типов
cout << "\nДинамическое приведение типов:" << endl;
Base* bptr = &derived;

// dynamic_cast возвращает nullptr при неудаче
Derived* dptr = dynamic_cast<Derived*>(bptr);
if(dptr) {
    cout << "Приведение успешно" << endl;
}

// Для ссылок dynamic_cast бросает исключение при неудаче
try {
    Derived& dref = dynamic_cast<Derived&>(*bptr);
    cout << "Приведение ссылки успешно" << endl;
} catch(const bad_cast& e) {
    cout << "Приведение не удалось: " << e.what() << endl;
}

```

```
    return 0;  
}
```

Часть 5: Практические задания

Задание 1: Реализация связного списка

```
#include <iostream>  
using namespace std;  
  
template<typename T>  
class LinkedList {  
private:  
    struct Node {  
        T data;  
        Node* next;  
        Node(T val) : data(val), next(nullptr) {}  
    };  
  
    Node* head;  
  
public:  
    LinkedList() : head(nullptr) {}  
  
    ~LinkedList() {  
        Node* current = head;  
        while(current) {  
            Node* next = current->next;  
            delete current;  
            current = next;  
        }  
    }  
  
    void add(T val) {  
        Node* new_node = new Node(val);  
        if(!head) {  
            head = new_node;  
        } else {
```

```

        Node* current = head;
        while(current->next) {
            current = current->next;
        }
        current->next = new_node;
    }
}

void print() const {
    Node* current = head;
    while(current) {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "nullptr" << endl;
}

// Другие методы: удаление, поиск, реверс и т.д.
};

int main() {
    LinkedList<int> list;
    list.add(1);
    list.add(2);
    list.add(3);
    list.print();

    LinkedList<string> str_list;
    str_list.add("Hello");
    str_list.add("World");
    str_list.print();

    return 0;
}

```

Задание 2: Умный массив с копированием при записи

```

#include <iostream>
#include <cstring>
using namespace std;

```

```

class SmartString {
private:
    struct StringData {
        char* data;
        size_t length;
        int ref_count;

        StringData(const char* str) {
            length = strlen(str);
            data = new char[length + 1];
            strcpy(data, str);
            ref_count = 1;
        }

        ~StringData() {
            delete[] data;
        }
    };

    StringData* ptr;

    void detach() {
        if(ptr && ptr->ref_count > 1) {
            ptr->ref_count--;
            ptr = new StringData(ptr->data);
        }
    }

public:
    SmartString(const char* str = "") : ptr(new StringData(str)) {}

    SmartString(const SmartString& other) : ptr(other.ptr) {
        ptr->ref_count++;
    }

    ~SmartString() {
        if(--ptr->ref_count == 0) {
            delete ptr;
        }
    }

    SmartString& operator=(const SmartString& other) {
        if(this != &other) {

```

```

        if(--ptr->ref_count == 0) {
            delete ptr;
        }
        ptr = other.ptr;
        ptr->ref_count++;
    }
    return *this;
}

char& operator[](size_t index) {
    detach();
    return ptr->data[index];
}

const char& operator[](size_t index) const {
    return ptr->data[index];
}

const char* c_str() const {
    return ptr->data;
}

int get_ref_count() const {
    return ptr->ref_count;
}
};

int main() {
    SmartString s1 = "Hello";
    SmartString s2 = s1;  // копирование, без дублирования данных

    cout << "После создания:" << endl;
    cout << "s1: " << s1.c_str() << " (refs: " << s1.get_ref_count() << endl;
    cout << "s2: " << s2.c_str() << " (refs: " << s2.get_ref_count() << endl;

    s2[0] = 'J';  // копирование при записи

    cout << "\nПосле модификации s2:" << endl;
    cout << "s1: " << s1.c_str() << " (refs: " << s1.get_ref_count() << endl;
    cout << "s2: " << s2.c_str() << " (refs: " << s2.get_ref_count() << endl;
}

```

```
    return 0;
}
```



Итоговая таблица сравнения

Характеристика	Указатели	Ссылки
Инициализация	Можно объявить без инициализации	Должны быть инициализированы сразу
nullptr	Могут быть nullptr	Не могут быть nullptr
Перенаправление	Можно перенаправить	Нельзя перенаправить после инициализации
Разыменование	Требуют * для доступа к значению	Не требуют разыменования
Арифметика	Поддерживают арифметику	Не поддерживают арифметику
Свой адрес	Имеют свой адрес (&ptr)	Не имеют своего адреса (&ref = адрес объекта)
Размер	sizeof(ptr) = размер указателя	sizeof(ref) = размер объекта
Массивы	int* ptr = arr	int (&ref)[N] = arr
const варианты	4 варианта (см. выше)	2 варианта (const &, &)
Безопасность	Менее безопасны	Более безопасны



Золотые правила

1. Используйте ссылки, когда:

- Параметр должен быть всегда валидным

- Нужно изменить оригинальный объект
- Хотите избежать копирования больших объектов
- Работаете с существующими переменными

2. Используйте указатели, когда:

- Параметр может отсутствовать (nullptr)
- Работаете с динамической памятью
- Нужна арифметика указателей
- Реализуете структуры данных
- Работаете с полиморфизмом

3. Всегда используйте `const` там, где это возможно

4. Используйте умные указатели вместо сырых для динамической памяти

5. Проверяйте указатели на nullptr перед использованием

6. Не возвращайте ссылки/указатели на локальные переменные

Это полное руководство покрывает все аспекты работы с указателями и ссылками в C++. Практикуйтесь с каждым примером, чтобы глубоко понять различия и правильно выбирать нужный инструмент для каждой задачи!

Задачи на функции и рекурсию для начинающих с подробным разбором

Задача 1: Базовые функции и параметры по значению

Условие:

Написать функцию для вычисления n-го числа Фибоначчи и функцию для проверки числа на простоту.

Разбор:

Что такое параметры по значению?

- При передаче параметра по значению создается копия переменной
- Изменения внутри функции не влияют на исходную переменную
- Хорошо подходит для базовых типов данных (int, double, char)

```
#include <iostream>
#include <cmath>
using namespace std;

// 1. Функция вычисления n-го числа Фибоначчи (параметры по значению)
int fibonacci(int n) {
    // Параметр n передается по значению - создается копия
    // Это безопасно, так как мы не меняем исходное значение

    // Базовые случаи для рекурсии
    if (n <= 0) return 0; // F(0) = 0
    if (n == 1) return 1; // F(1) = 1

    // Рекурсивный вызов
    // Каждый вызов получает свою копию параметра
    return fibonacci(n - 1) + fibonacci(n - 2);

    // Проблема: это неэффективная реализация O(2^n)
    // F(4) вычисляется как:
    // F(4) = F(3) + F(2)
    // F(3) = F(2) + F(1)
    // F(2) = F(1) + F(0) = 1 + 0 = 1
    // И так далее...
}

// 2. Функция проверки числа на простоту (параметр по значению)
bool isPrime(int number) {
    // number передается по значению - создается копия
    // Мы можем безопасно изменять number внутри функции

    if (number <= 1) return false; // Простые числа начинаются с 2
    if (number == 2) return true; // 2 - единственное четное простое число
    if (number % 2 == 0) return false; // Остальные четные числа не простые
```

```

// Проверяем делители до квадратного корня из числа
// Используем i += 2 чтобы проверять только нечетные делители
for (int i = 3; i <= sqrt(number); i += 2) {
    if (number % i == 0) {
        return false; // Нашли делитель - число не простое
    }
}

return true; // Делителей не нашлось - число простое
}

// 3. Улучшенная версия Фибоначчи с мемоизацией (статическая инициализация)
int fibonacciFast(int n) {
    // Статический массив для хранения уже вычисленных значений
    static int мемо[100] = {0};

    // Инициализируем базовые случаи
    if (n <= 0) return 0;
    if (n == 1) return 1;

    // Если значение уже вычислено, возвращаем его
    if (мемо[n] != 0) {
        return мемо[n];
    }

    // Вычисляем и сохраняем результат
    мемо[n] = fibonacciFast(n - 1) + fibonacciFast(n - 2);
    return мемо[n];
}

int main() {
    // Демонстрация работы с параметрами по значению

    int num = 10; // Исходная переменная

    cout << "=== Числа Фибоначчи ===" << endl;
    for (int i = 0; i <= num; i++) {
        cout << "F(" << i << ") = " << fibonacci(i) << endl;
        // i передается по значению - fibonacci получает копию
        // Исходное i не меняется
    }
}

```

```

cout << "\n=== Быстрое вычисление ===" << endl;
cout << "F(20) = " << fibonacciFast(20) << endl;
cout << "F(30) = " << fibonacciFast(30) << endl;

cout << "\n=== Проверка на простоту ===" << endl;
int testNumbers[] = {1, 2, 3, 4, 17, 25, 29, 100};

for (int i = 0; i < 8; i++) {
    int currentNum = testNumbers[i];
    bool prime = isPrime(currentNum);
    // currentNum передается по значению в isPrime

    cout << currentNum << " - " << (prime ? "простое" : "сложное") << endl;
}

// Важно: параметры по значению защищают исходные данные
int original = 5;
int result = fibonacci(original);
cout << "\noriginal = " << original << ", result = " << result << endl;
// original осталось 5, несмотря на все вычисления в fibonacci

return 0;
}

```

Задача 2: Параметры по ссылке и swap

Условие:

Реализовать функцию swap для обмена значений двух переменных и функцию для подсчета суммы цифр числа с накоплением результата.

Разбор:

Что такое параметры по ссылке?

- Передается ссылка на исходную переменную, а не копия
- Изменения внутри функции влияют на исходную переменную
- Эффективно для больших объектов (экономит память)

```
#include <iostream>
using namespace std;

// 1. Классическая функция swap (параметры по ссылке)
void swapValues(int& a, int& b) {
    // & означает "ссылка на"
    // a и b - это псевдонимы для исходных переменных
    // Все изменения a и b изменяют исходные переменные

    int temp = a; // temp получает значение, на которое ссылается a
    a = b;         // a теперь ссылается на значение b
    b = temp;      // b теперь ссылается на старое значение a

    // Пример: если вызвать swapValues(x, y) где x=5, y=10
    // temp = 5 (значение x)
    // x = 10 (теперь x равно 10)
    // y = 5 (теперь y равно 5)
}

// 2. НЕПРАВИЛЬНАЯ версия swap (параметры по значению)
void swapWrong(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    // Здесь меняются только локальные копии!
    // Исходные переменные не изменятся
}

// 3. Функция для разворота массива (параметры по ссылке)
void reverseArray(int arr[], int size) {
    // arr[] передается как указатель, но мы работаем с исходным массивом
    for (int i = 0; i < size / 2; i++) {
        swapValues(arr[i], arr[size - 1 - i]);
    }
}

// 4. Функция для подсчета суммы цифр с накоплением (ссылка на сумму)
void sumOfDigits(int number, int& sum) {
    // number - по значению (копия, можно изменять)
    // sum - по ссылке (будет накапливать результат)

    sum = 0; // Инициализируем sum (это изменяет исходную переменную)
```

```
// Обрабатываем отрицательные числа
if (number < 0) {
    number = -number; // Работаем с модулем
}

// Суммируем цифры
while (number > 0) {
    int digit = number % 10; // Последняя цифра
    sum += digit;           // Добавляем к сумме
    number /= 10;          // Убираем последнюю цифру
}

// Пример: number = 123
// 1 итерация: digit = 3, sum = 3, number = 12
// 2 итерация: digit = 2, sum = 5, number = 1
// 3 итерация: digit = 1, sum = 6, number = 0
}

// 5. Функция для подсчета суммы цифр (возвращаемое значение)
int sumOfDigitsReturn(int number) {
    int sum = 0;

    if (number < 0) {
        number = -number;
    }

    while (number > 0) {
        sum += number % 10;
        number /= 10;
    }

    return sum; // Возвращаем копию значения
}

// 6. Функция с несколькими результатами (ссылки для возврата
void minMaxSum(int arr[], int size, int& minVal, int& maxVal,
    if (size == 0) return;

    minVal = arr[0];
    maxVal = arr[0];
    sum = 0;
```

```
    for (int i = 0; i < size; i++) {
        if (arr[i] < minVal) minVal = arr[i];
        if (arr[i] > maxVal) maxVal = arr[i];
        sum += arr[i];
    }
}

int main() {
    cout << "=== Демонстрация swap ===" << endl;

    int x = 5, y = 10;
    cout << "До swap: x = " << x << ", y = " << y << endl;

    swapValues(x, y); // Правильный swap
    cout << "После swap: x = " << x << ", y = " << y << endl;

    // Попробуем неправильную версию
    swapWrong(x, y); // Ничего не изменится!
    cout << "После swapWrong: x = " << x << ", y = " << y << endl;

    cout << "\n=== Разворот массива ===" << endl;
    int arr[] = {1, 2, 3, 4, 5};
    int size = 5;

    cout << "Исходный массив: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    reverseArray(arr, size);

    cout << "Развернутый массив: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    cout << "\n=== Сумма цифр числа ===" << endl;

    int number = 12345;
    int sum1 = 0, sum2 = 0;
```

```
// Версия с ссылкой
sumOfDigits(number, sum1);
cout << "Сумма цифр " << number << " (ссылка): " << sum1 .

// Версия с возвращаемым значением
sum2 = sumOfDigitsReturn(number);
cout << "Сумма цифр " << number << " (возврат): " << sum2

cout << "\n=== Несколько результатов через ссылки ===" <<
int testArr[] = {3, 1, 4, 1, 5, 9, 2, 6};
int arrSize = 8;
int minVal, maxVal, totalSum;

minMaxSum(testArr, arrSize, minVal, maxVal, totalSum);

cout << "Массив: ";
for (int i = 0; i < arrSize; i++) {
    cout << testArr[i] << " ";
}
cout << endl;

cout << "Минимум: " << minVal << endl;
cout << "Максимум: " << maxVal << endl;
cout << "Сумма: " << totalSum << endl;
cout << "Среднее: " << (double)totalSum / arrSize << endl

return 0;
}
```

Задача 3: Рекурсия и факториал (задача с собеседования)

Условие:

Реализовать вычисление факториала рекурсивно и итеративно. Добавить подсчет количества вызовов.

Разбор:

Что такое рекурсия?

- Функция вызывает саму себя
- Должен быть базовый случай (когда рекурсия останавливается)
- Каждый вызов уменьшает проблему

```
#include <iostream>
using namespace std;
```

```
// Глобальная переменная для подсчета вызовов (обычно так не ,
int callCount = 0;
```

```
// 1. Классическая рекурсивная версия факториала
```

```
long long factorialRecursive(int n) {
    callCount++; // Увеличиваем счетчик вызовов
```

```
    // Базовый случай:  $0! = 1$ ,  $1! = 1$ 
```

```
    if (n <= 1) {
        return 1;
    }
```

```
    // Рекурсивный случай:  $n! = n * (n-1)!$ 
    return n * factorialRecursive(n - 1);
```

```
    // Пример для  $n = 4$ :
```

```
    //  $\text{factorial}(4) = 4 * \text{factorial}(3)$ 
```

```
    //  $\text{factorial}(3) = 3 * \text{factorial}(2)$ 
```

```
    //  $\text{factorial}(2) = 2 * \text{factorial}(1)$ 
```

```
    //  $\text{factorial}(1) = 1$ 
```

```
    // Затем раскручиваем стек:
```

```
    //  $\text{factorial}(2) = 2 * 1 = 2$ 
```

```
    //  $\text{factorial}(3) = 3 * 2 = 6$ 
```

```
    //  $\text{factorial}(4) = 4 * 6 = 24$ 
```

```
}
```

```
// 2. Итеративная версия факториала (цикл)
```

```
long long factorialIterative(int n) {
    long long result = 1;
```

```
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
```

```
        return result;
    }

// 3. Рекурсия с хвостовой оптимизацией (теоретически)
long long factorialTailRecursive(int n, long long accumulator,
    callCount++);

// Базовый случай
if (n <= 1) {
    return accumulator;
}

// Хвостовая рекурсия: результат накапливается в параметрах
return factorialTailRecursive(n - 1, n * accumulator);

// Компиляторы C++ обычно не оптимизируют хвостовую рекурсию
// но такой подход полезен для понимания
}

// 4. Рекурсия для возведения в степень
double power(double base, int exponent) {
    // Базовый случай
    if (exponent == 0) {
        return 1.0;
    }

    // Если степень отрицательная
    if (exponent < 0) {
        return 1.0 / power(base, -exponent);
    }

    // Рекурсивный случай
    return base * power(base, exponent - 1);
}

// 5. Оптимизированное возведение в степень (O(log n))
double powerFast(double base, int exponent) {
    // Базовый случай
    if (exponent == 0) {
        return 1.0;
    }
}
```

```
// Если степень отрицательная
if (exponent < 0) {
    return 1.0 / powerFast(base, -exponent);
}

// Если степень четная
if (exponent % 2 == 0) {
    double half = powerFast(base, exponent / 2);
    return half * half;
}
// Если степень нечетная
else {
    return base * powerFast(base, exponent - 1);
}

// Пример: 2^10
// 2^10 = (2^5)^2
// 2^5 = 2 * 2^4
// 2^4 = (2^2)^2
// 2^2 = (2^1)^2
// 2^1 = 2 * 2^0 = 2 * 1 = 2
// Затем поднимаемся обратно
}

// 6. Рекурсивная функция для суммы массива
int arraySumRecursive(int arr[], int size) {
    // Базовый случай: пустой массив
    if (size == 0) {
        return 0;
    }

    // Рекурсивный случай: последний элемент + сумма остальных
    return arr[size - 1] + arraySumRecursive(arr, size - 1);

    // Пример: arr = [1, 2, 3], size = 3
    // arraySum([1,2,3], 3) = 3 + arraySum([1,2], 2)
    // arraySum([1,2], 2) = 2 + arraySum([1], 1)
    // arraySum([1], 1) = 1 + arraySum([], 0)
    // arraySum([], 0) = 0
    // Итого: 1 + 2 + 3 + 0 = 6
}

// 7. Поиск максимального элемента в массиве рекурсивно
```

```

int findMaxRecursive(int arr[], int size) {
    // Базовый случай: один элемент
    if (size == 1) {
        return arr[0];
    }

    // Рекурсивный случай: сравниваем первый элемент с максимумом
    int maxOfRest = findMaxRecursive(arr + 1, size - 1);
    return (arr[0] > maxOfRest) ? arr[0] : maxOfRest;

    // arr + 1 - это указатель на следующий элемент
    // Пример: arr = [3, 1, 4, 2], size = 4
    // max([3,1,4,2], 4) = max(3, max([1,4,2], 3))
    // max([1,4,2], 3) = max(1, max([4,2], 2))
    // max([4,2], 2) = max(4, max([2], 1))
    // max([2], 1) = 2
    // max(4, 2) = 4
    // max(1, 4) = 4
    // max(3, 4) = 4
}

int main() {
    cout << "=== Факториал рекурсивно ===" << endl;

    for (int i = 0; i <= 10; i++) {
        callCount = 0;
        long long result = factorialRecursive(i);
        cout << i << "! = " << result << " (вызовов: " << callCount << endl;
    }

    cout << "\n=== Факториал итеративно ===" << endl;
    for (int i = 0; i <= 10; i++) {
        cout << i << "! = " << factorialIterative(i) << endl;
    }

    cout << "\n=== Факториал с хвостовой рекурсией ===" << endl;
    callCount = 0;
    cout << "5! = " << factorialTailRecursive(5) << " (вызовов: " << callCount << endl;

    cout << "\n=== Возведение в степень ===" << endl;
    double base = 2.0;

    for (int exp = -3; exp <= 3; exp++) {

```

```

        cout << base << "^" << exp << " = " << power(base, exp);
        cout << " (быстро: " << powerFast(base, exp) << ")" << endl;
    }

    cout << "\n=== Сумма массива рекурсивно ===" << endl;
    int arr[] = {1, 2, 3, 4, 5};
    int size = 5;

    cout << "Массив: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    int sum = arraySumRecursive(arr, size);
    cout << "Сумма элементов: " << sum << endl;

    cout << "\n=== Максимум в массиве рекурсивно ===" << endl;
    int arr2[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int size2 = 8;

    cout << "Массив: ";
    for (int i = 0; i < size2; i++) {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int maxVal = findMaxRecursive(arr2, size2);
    cout << "Максимальный элемент: " << maxVal << endl;

    // Важное замечание о рекурсии
    cout << "\n=== Предупреждение о глубокой рекурсии ===" << endl;

    try {
        // Факториал 20 уже очень большое число
        cout << "20! = " << factorialRecursive(20) << endl;

        // Факториал больших чисел может вызвать переполнение
        // cout << "10000! - это вызовет переполнение стека!"
        // factorialRecursive(10000); // ОПАСНО!
    } catch (...) {
        cout << "Произошло переполнение стека!" << endl;
    }
}

```

```
        return 0;  
    }
```

Задача 4: Алгоритмы с собеседований (бинарный поиск, палиндромы)

Условие:

Реализовать бинарный поиск рекурсивно и итеративно, проверить строку на палиндром.

Разбор:

Задачи с реальных собеседований:

- Бинарный поиск - основа многих алгоритмов
- Палиндромы - проверка симметричности
- Работа со строками и массивами

```
#include <iostream>  
#include <string>  
#include <cctype> // для tolower()  
using namespace std;  
  
// 1. Бинарный поиск (итеративная версия)  
int binarySearchIterative(int arr[], int size, int target) {  
    int left = 0;  
    int right = size - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2; // Предотвращает  
  
        if (arr[mid] == target) {  
            return mid; // Нашли элемент  
        } else if (arr[mid] < target) {  
            left = mid + 1; // Ищем в правой половине
```

```
        } else {
            right = mid - 1; // Ищем в левой половине
        }
    }

    return -1; // Элемент не найден
}

// 2. Бинарный поиск (рекурсивная версия)
int binarySearchRecursive(int arr[], int left, int right, int target) {
    // Базовый случай: интервал пуст
    if (left > right) {
        return -1;
    }

    int mid = left + (right - left) / 2;

    // Базовый случай: нашли элемент
    if (arr[mid] == target) {
        return mid;
    }

    // Рекурсивные случаи
    if (arr[mid] < target) {
        // Ищем в правой половине
        return binarySearchRecursive(arr, mid + 1, right, target);
    } else {
        // Ищем в левой половине
        return binarySearchRecursive(arr, left, mid - 1, target);
    }
}

// 3. Проверка строки на палиндром (рекурсивно)
bool isPalindromeRecursive(const string& str, int start, int end) {
    // Базовый случай: строка длиной 0 или 1 символа
    if (start >= end) {
        return true;
    }

    // Если символы на концах не совпадают
    if (str[start] != str[end]) {
        return false;
    }
}
```

```
    // Рекурсивно проверяем подстроку без крайних символов
    return isPalindromeRecursive(str, start + 1, end - 1);
}
```

```
// 4. Проверка строки на палиндром (итеративно)
bool isPalindromeIterative(const string& str) {
    int start = 0;
    int end = str.length() - 1;

    while (start < end) {
        if (str[start] != str[end]) {
            return false;
        }
        start++;
        end--;
    }

    return true;
}
```

```
// 5. Проверка палиндрома без учета регистра и пробелов
bool isPalindromeAdvanced(const string& str) {
    int start = 0;
    int end = str.length() - 1;

    while (start < end) {
        // Пропускаем не-буквенные символы слева
        while (start < end && !isalnum(str[start])) {
            start++;
        }

        // Пропускаем не-буквенные символы справа
        while (start < end && !isalnum(str[end])) {
            end--;
        }

        // Сравниваем символы без учета регистра
        if (tolower(str[start]) != tolower(str[end])) {
            return false;
        }

        start++;
    }
}
```

```
        end--;  
    }  
  
    return true;  
}  
  
// 6. Нахождение наибольшего общего делителя (НОД) рекурсивно  
int gcdRecursive(int a, int b) {  
    // Алгоритм Евклида  
    // Базовый случай: если b = 0, то НОД = a  
    if (b == 0) {  
        return a;  
    }  
  
    // Рекурсивный случай: НОД(a, b) = НОД(b, a % b)  
    return gcdRecursive(b, a % b);  
  
    // Пример: НОД(48, 18)  
    // gcd(48, 18) = gcd(18, 48 % 18 = 12)  
    // gcd(18, 12) = gcd(12, 18 % 12 = 6)  
    // gcd(12, 6) = gcd(6, 12 % 6 = 0)  
    // gcd(6, 0) = 6  
}  
  
// 7. Нахождение НОД итеративно  
int gcdIterative(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}  
  
// 8. Реверс строки рекурсивно  
void reverseStringRecursive(string& str, int start, int end) {  
    // Базовый случай  
    if (start >= end) {  
        return;  
    }  
  
    // Меняем символы местами  
    swap(str[start], str[end]);  
}
```

```

        // Рекурсивно реверсируем подстроку
        reverseStringRecursive(str, start + 1, end - 1);
    }

// 9. Вывод всех подмножеств массива (рекурсивно)
void printSubsets(int arr[], int size, int index, string current)
{
    // Базовый случай: дошли до конца массива
    if (index == size) {
        cout << "{" << current << "}" << endl;
        return;
    }

    // Рекурсивные случаи:
    // 1. Не включаем текущий элемент
    printSubsets(arr, size, index + 1, current);

    // 2. Включаем текущий элемент
    string newCurrent = current;
    if (!current.empty()) {
        newCurrent += ", ";
    }
    newCurrent += to_string(arr[index]);
    printSubsets(arr, size, index + 1, newCurrent);
}

int main() {
    cout << "=== Бинарный поиск ===" << endl;

    int sortedArr[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
    int arrSize = 10;

    cout << "Массив: ";
    for (int i = 0; i < arrSize; i++) {
        cout << sortedArr[i] << " ";
    }
    cout << endl;

    int target = 7;
    int resultIterative = binarySearchIterative(sortedArr, arrSize, target);
    int resultRecursive = binarySearchRecursive(sortedArr, 0, arrSize - 1, target);

    cout << "Ищем " << target << ":" << endl;
}

```

```
cout << "Итеративно: индекс " << resultIterative << endl;
cout << "Рекурсивно: индекс " << resultRecursive << endl;

target = 8; // Не существует в массиве
resultIterative = binarySearchIterative(sortedArr, arrSize, target);
cout << "Ищем " << target << ": индекс " << resultIterative << endl;

cout << "\n=== Проверка палиндромов ===" << endl;

string testStrings[] = {"radar", "hello", "A man a plan a canal panama"};

for (const string& str : testStrings) {
    bool simple = isPalindromeIterative(str);
    bool recursive = isPalindromeRecursive(str, 0, str.length() - 1);
    bool advanced = isPalindromeAdvanced(str);

    cout << "\"" << str << "\": " << endl;
    cout << "    Простая проверка: " << (simple ? "палиндром" : "не палиндром") << endl;
    cout << "    Рекурсивная: " << (recursive ? "палиндром" : "не палиндром") << endl;
    cout << "    Продвинутая: " << (advanced ? "палиндром" : "не палиндром") << endl;
}

cout << "\n=== Наибольший общий делитель ===" << endl;

int pairs[][2] = {{48, 18}, {1071, 462}, {17, 13}, {100, 342}};

for (int i = 0; i < 4; i++) {
    int a = pairs[i][0];
    int b = pairs[i][1];

    int gcdR = gcdRecursive(a, b);
    int gcdI = gcdIterative(a, b);

    cout << "НОД(" << a << ", " << b << "): " << gcdR << endl;
    cout << "    Рекурсивно: " << gcdR << endl;
    cout << "    Итеративно: " << gcdI << endl;
}

cout << "\n=== Реверс строки рекурсивно ===" << endl;

string str = "Hello, World!";
cout << "Исходная строка: " << str << endl;
```

```

reverseStringRecursive(str, 0, str.length() - 1);
cout << "Реверсированная: " << str << endl;

cout << "\n=== Все подмножества массива ===" << endl;

int smallArr[] = {1, 2, 3};
int smallSize = 3;

cout << "Массив: {1, 2, 3}" << endl;
cout << "Все подмножества:" << endl;
printSubsets(smallArr, smallSize, 0);

// Важные замечания для собеседований
cout << "\n=== Советы для собеседований ===" << endl;
cout << "1. Бинарный поиск: всегда проверяйте corner cases" << endl;
cout << "    - Пустой массив" << endl;
cout << "    - Один элемент" << endl;
cout << "    - Элемент не существует" << endl;
cout << "    - Все элементы одинаковые" << endl;

cout << "\n2. Рекурсия: помните о стеке!" << endl;
cout << "    - Глубокая рекурсия может вызвать Stack Overflow" << endl;
cout << "    - Всегда проверяйте базовый случай" << endl;
cout << "    - Рассмотрите итеративную альтернативу" << endl;

cout << "\n3. Строки: учитывайте регистр и пробелы!" << endl;
cout << "    - tolower() для игнорирования регистра" << endl;
cout << "    - isalnum() для проверки алфавитно-цифровых символов" << endl;
cout << "    - Пустая строка - это палиндром" << endl;

return 0;
}

```

Ключевые выводы для новичков:

1. Параметры по значению vs по ссылке:

- **По значению:** безопасно, но медленно для больших объектов
- **По ссылке:** эффективно, но можно случайно изменить исходные данные

- **Константные ссылки:** безопасно и эффективно для чтения больших объектов

2. Рекурсия:

- Всегда должен быть **базовый случай**
- Каждый рекурсивный вызов должен **уменьшать задачу**
- Помните о **глубине рекурсии** и стеке
- Для больших n используйте **итеративные решения**

3. Возвращаемые значения:

- Может возвращать **один** результат
- Для нескольких результатов используйте **ссылки или указатели**
- **void** функции не возвращают значения

4. Советы для собеседований:

- Всегда проверяйте **крайние случаи** (empty, null, 0, 1)
- Объясняйте **сложность алгоритма** ($O(n)$, $O(\log n)$)
- Предлагайте **оптимизации**
- Пишите **чистый, читаемый код**
- Комментируйте **ход своих мыслей**

Эти задачи охватывают все основные аспекты функций и рекурсии в C++, которые часто встречаются на собеседованиях для начинающих разработчиков.